AMITY UNIVERSITY
UTTAR PRADESH

# LAB RECORD

## BACHELOR OF TECHNOLOGY

B.Tech. CS&E _____ Semester ( VI )

**(Academic Session – 2021 - 22 )**

| | | |
|---|---|---|
| Course Title | | : Compiler Construction |
| Course Code | : | CSE304 |
| Enrollment No. | | : A7605219046 |
| Name of Student | | : Anshika Jain |
| Date of Submission | | : __ __ / __ __ /__ __ __ __ |
| Signature of Student | | : |
| Grade/Marks Obtained | : | |
| Faculty Name & Signature | | : Dr. Pawan Singh |

**Department of Computer Science & Engineering**
**Amity School of Engineering & Technology**
**Amity University, Lucknow Campus**

Department of Computer Science & Engineering
Amity School of Engineering & Technology
Amity University, Lucknow Campus

# Index

| Lab Session | Date | Signature of Faculty | Remarks |
|---|---|---|---|
| 1.Write a program in C/C++ to accept the regular expression- (abb*). | | | |
| 2. Write a program in C/C++ to accept the regular expression- abb+cd+ | | | |
| 3. Write a program in C/C++ to recognize a token (RELOP). | | | |
| 4. Write a program in C/C++ to design a lexical analyzer to identify integer, operators, keywords. | | | |
| 5. | | | |
| 6. | | | |
| Q7. | | | |
| Q8 | | | |
| Q9. | | | |
| 10. | | | |

**Aim:-** Write a program to accept regular expression abb+cd+

**Software used:-** onlinegdb.com (Online compiler)

**Procedure:-** Go to debug -> run or press CTRL + F9 to run the program.

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<stdlib.h>
int main()
{
char s[20],c;
int flag=0,i=0;

printf("\n Enter a string:");
gets(s);
while(s[i]!='\0')
{
switch(flag)
{
case 0: c=s[i++];
if(c=='a')
flag=1;
else
flag = 5 ;
break;
case 1: c=s[i++];
if(c=='b')
flag=2;
else
flag = 5;
```

```
break;
case 2 : c=s[i++];
if(c=='b')
flag = 2;
else if (c=='c')
flag = 3;
else
flag = 5;
break;
case 3 : c=s[i++];
if(c=='d')
flag = 4;
else
flag=5;
break;
exit(0);
}
}
if (flag==5)
printf ("Rejected");
else
printf ("Accepted");
return 0;
}
```

Output:

```
 Enter a string:abbcd
Accepted

...Program finished with exit code 0
ess ENTER to exit console.
```

**Aim:- Write a program in C/C++ to recognize a token (RELOP).**

**Software used:- onlinegdb.com (Online compiler)**

**Procedure:- Go to debug -> run or press CTRL + F9 to run the program.**

**Program:-**

```
#include<bits/stdc++.h>
using namespace std;
struct TOKEN {
  std::string token;
  std::string attribute;
};
void fail() {
  cout << "Lexical Error: Invalid character" << "\n";
}
TOKEN ques(string s) {
  TOKEN t;

  int i = 0, state = 0;
  while(true) {
    char c = s[i];
    switch(state){
      case 0: {
        if (c == '<') state = 1;
        else if (c == '=') state = 5;
        else if (c == '>') state = 6;
        else fail();
        break;
```

```
}
case 1: {
  if(c == '=') state = 2;
  else if(c == '>') state = 3;
  else state = 4;
  break;
}
case 2: {
  t.token = "RELOP";
  t.attribute = "LE";
  return t;
}
case 3: {
  t.token = "RELOP";
  t.attribute = "NE";
  return t;
}
case 4: {
  t.token = "RELOP";
  t.attribute = "LT";
  return t;
}
case 5: {
  t.token = "RELOP";
  t.attribute = "EQ";
  return t;
}
case 6: {
  if(c == '=') state = 7;
  else state = 8;
```

```
      break;
    }
    case 7: {
      t.token = "RELOP";
      t.attribute = "GE";
      return t;
    }
    case 8: {
      t.token = "RELOP";
      t.attribute = "GT";
      return t;
    }
  }
  ++i;
 }
 return t;
}

int main() {
 string s;
 cin >> s;
 TOKEN p = ques(s);
 cout << p.token << " " << p.attribute << "\n";
 return 0;
}
```

Input & Output:

```
<
RELOP LT
```

**Aim-: Write a program in C/C++ to design a lexical analyzer to identify integer, operators, keywords.**

**Software used: - onlinegdb.com (Online compiler)**

**Procedure: - Go to debug -> run or press CTRL + F9 to run the program.**

**Program Logic: -**

1. Read the input Expression.
2. Check whether input is alphabet or digits then store it as identifier.
3. If the input is operator store it as symbol
4. Check the input for keywords.

Program: -

```
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

bool isDelimiter(char ch)
{
	if (ch == ' ' || ch == '+' || ch == '-' || ch == '*' ||
		ch == '/' || ch == ',' || ch == ';' || ch == '>' ||
		ch == '<' || ch == '=' || ch == '(' || ch == ')' ||
		ch == '[' || ch == ']' || ch == '{' || ch == '}')
		return (true);
```

```
        return (false);
}

bool isOperator(char ch)
{
        if (ch == '+' || ch == '-' || ch == '*' ||
                ch == '/' || ch == '>' || ch == '<' ||
                ch == '=')
                return (true);
        return (false);
}

bool validIdentifier(char* str)
{
        if (str[0] == '0' || str[0] == '1' || str[0] == '2' ||
                str[0] == '3' || str[0] == '4' || str[0] == '5' ||
                str[0] == '6' || str[0] == '7' || str[0] == '8' ||
                str[0] == '9' || isDelimiter(str[0]) == true)
                return (false);
        return (true);
}

bool isKeyword(char* str)
{
        if (!strcmp(str, "if") || !strcmp(str, "else") ||
                !strcmp(str, "while") || !strcmp(str, "do") ||
                !strcmp(str, "break") ||
                !strcmp(str, "continue") || !strcmp(str, "int")
                || !strcmp(str, "double") || !strcmp(str, "float")
                || !strcmp(str, "return") || !strcmp(str, "char")
```

```
		|| !strcmp(str, "case") || !strcmp(str, "char")
		|| !strcmp(str, "sizeof") || !strcmp(str, "long")
		|| !strcmp(str, "short") || !strcmp(str, "typedef")
		|| !strcmp(str, "switch") || !strcmp(str, "unsigned")
		|| !strcmp(str, "void") || !strcmp(str, "static")
		|| !strcmp(str, "struct") || !strcmp(str, "goto"))
		return (true);
	return (false);
}

bool isInteger(char* str)
{
	int i, len = strlen(str);

	if (len == 0)
		return (false);
	for (i = 0; i < len; i++) {
		if (str[i] != '0' && str[i] != '1' && str[i] != '2'
			&& str[i] != '3' && str[i] != '4' && str[i] != '5'
			&& str[i] != '6' && str[i] != '7' && str[i] != '8'
			&& str[i] != '9' || (str[i] == '-' && i > 0))
			return (false);
	}
	return (true);
}

// Returns 'true' if the string is a REAL NUMBER.
bool isRealNumber(char* str)
{
	int i, len = strlen(str);
```

```
	bool hasDecimal = false;

	if (len == 0)
		return (false);
	for (i = 0; i < len; i++) {
		if (str[i] != '0' && str[i] != '1' && str[i] != '2'
			&& str[i] != '3' && str[i] != '4' && str[i] != '5'
			&& str[i] != '6' && str[i] != '7' && str[i] != '8'
			&& str[i] != '9' && str[i] != '.' ||
			(str[i] == '-' && i > 0))
			return (false);
		if (str[i] == '.')
			hasDecimal = true;
	}
	return (hasDecimal);
}

char* subString(char* str, int left, int right)
{
	int i;
	char* subStr = (char*)malloc(
				sizeof(char) * (right - left + 2));

	for (i = left; i <= right; i++)
		subStr[i - left] = str[i];
	subStr[right - left + 1] = '\0';
	return (subStr);
}

void parse(char* str)
```

```
{
        int left = 0, right = 0;
        int len = strlen(str);

        while (right <= len && left <= right) {
                if (isDelimiter(str[right]) == false)
                        right++;

                if (isDelimiter(str[right]) == true && left == right) {
                        if (isOperator(str[right]) == true)
                                printf("'%c' IS AN OPERATOR\n", str[right]);

                        right++;
                        left = right;
                } else if (isDelimiter(str[right]) == true && left != right
                                || (right == len && left != right)) {
                        char* subStr = subString(str, left, right - 1);

                        if (isKeyword(subStr) == true)
                                printf("'%s' IS A KEYWORD\n", subStr);

                        else if (isInteger(subStr) == true)
                                printf("'%s' IS AN INTEGER\n", subStr);

                        else if (isRealNumber(subStr) == true)
                                printf("'%s' IS A REAL NUMBER\n", subStr);

                        else if (validIdentifier(subStr) == true
                                        && isDelimiter(str[right - 1]) == false)
                                printf("'%s' IS A VALID IDENTIFIER\n", subStr);
```

```
                    else if (validIdentifier(subStr) == false
                                && isDelimiter(str[right - 1]) == false)
                        printf("'%s' IS NOT A VALID IDENTIFIER\n", subStr);
                    left = right;
            }
        }
        return;
}

]int main()
{
        // maximum legth of string is 100 here
        char str[100] = "int a = b + 1c; ";

        parse(str); // calling the parse function

        return (0);
}
```

Input & Output:

```
'int' IS A KEYWORD
'a' IS A VALID IDENTIFIER
'=' IS AN OPERATOR
'b' IS A VALID IDENTIFIER
'+' IS AN OPERATOR
'1c' IS NOT A VALID IDENTIFIER
```

```
#include<stdio.h>
#include<string.h>
#include<conio.h>
void main()  {
    char input[100],l[50],r[50],temp[10],tempprod[20],productions[25][50];
    int i=0,j=0,flag=0,consumed=0;
    clrscr();
    printf("Enter the productions: ");
    scanf("%1s->%s",l,r);
    printf("%s",r);
    while(sscanf(r+consumed,"%[^|]s",temp) == 1 && consumed <= strlen(r))  {
        if(temp[0] == l[0])  {
            flag = 1;
            sprintf(productions[i++],"%s->%s%s'\0",l,temp+1,l);
        }
        else
            sprintf(productions[i++],"%s'->%s%s'\0",l,temp,l);
        consumed += strlen(temp)+1;
    }
    if(flag == 1)  {
        sprintf(productions[i++],"%s->ε\0",l);
        printf("The productions after eliminating Left Recursion are:\n");
        for(j=0;j<i;j++)
            printf("%s\n",productions[j]);
    }
    else
        printf("The Given Grammar has no Left Recursion");
    getch();
}
```

```
#include<stdio.h>
#include<string.h>
#include<conio.h>
#define SIZE 10
  int main () {
      char non_terminal;
      char beta,alpha;
      int num, i;
      char production[10][SIZE];
      int index=3;
      clrscr();
      printf("Enter Number of Production : ");
      scanf("%d",&num);
      printf("Enter the grammar as E->E-A :\n");
      for(i=0;i<num;i++){
              scanf("%s",production[i]);
      }
      for(i=0;i<num;i++){
              printf("\nGRAMMAR : : : %s",production[i]);
              non_terminal=production[i][0];
              if(non_terminal==production[i][index]) {
                      alpha=production[i][index+1];
                      printf(" is left recursive.\n");
                      while(production[i][index]!=0 && production[i][index]!='|')
                          index++;
                      if(production[i][index]!=0) {
                          beta=production[i][index+1];
                          printf("Grammar without left recursion:\n");
                          printf("%c->%c%c\'",non_terminal,beta,non_terminal);
                          printf("\n%c\'->%c%c\'|E\n",non_terminal,alpha,non_terminal);
```

```
                }
                else
                    printf(" can't be reduced\n");
            }
            else
                printf(" is not left recursive.\n");
            index=3;
            getch();
        }
    }
```